

Bonus Assignment

Zain ul abideen
2440818.

Chapter : 9

B.C.S - 4F

9.4)

$$\text{Pages} = 64 = 2^6$$

$$\text{Page size} = 1024 = 2^{10}$$

$$\text{frames} = 32 = 2^5$$

(a) logical address :-

$$2^6 \times 2^{10} = 2^{16} = 16 \text{ bits}$$

(b) Physical address :-

$$2^5 \times 2^{10} = 2^{15} = 15 \text{ bits}$$

9.5)

115 KB, 500 KB, 358 KB, 200 KB, 375 KB

	First-fit	Best-fit	Worst-fit
300 KB	115 KB		
600 KB	500 KB	500 KB	358 KB
350 KB	200 KB		200 KB
200 KB		200 KB	
750 KB	358 KB	358 KB	115 KB
	375 KB	375 KB	500 KB
125 KB		115 KB	

9.7)

Page Size = 1 KB

Page Numbers = ?

offsets = ?

(a) 3085

$$\begin{aligned}\text{Page number} &= \text{logical address} / \text{Page size} \\ &= 3085 / 1024 \\ &= 3\end{aligned}$$

$$\begin{aligned}\text{Offset} &= \text{logical address} \bmod \text{Page size} \\ &= 3085 \bmod 1024 \\ &= 10\end{aligned}$$

(b) 42095

$$\begin{aligned}\text{Page Number} &= 42095 / 1024 = \\ &= 41\end{aligned}$$

$$\begin{aligned}\text{Offset} &= 42095 \bmod 1024 \\ &= 11\end{aligned}$$

(c) 215201

$$\begin{aligned}\text{Page Number} &= 215201 / 1024 \\ &= 210\end{aligned}$$

$$\begin{aligned}\text{Offset} &= 215201 \bmod 1024 \\ &= 161\end{aligned}$$

(d) 650000

$$\begin{aligned}\text{Page Number} &= 634 \\ \text{offset} &= 784\end{aligned}$$

(R) 2000001

Page Number = 1953

Offsets = 129

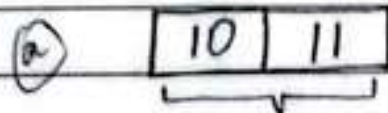
9.8)

virtual address = 21 bit

Physical address = 16 bit

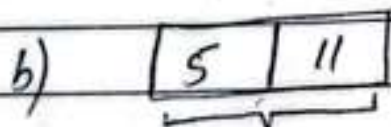
Page size = 2 KB

$$\text{Offset} = \log_2(2048) = 11$$



21 bit

A conventional single level page table have entries $2^{10} = 1 \text{ KB}$



16 bit

So, an inverted page table have entries 2^5

The maximum amount of physical address is $2^{16} = 64 \text{ KB}$

9.9)

$$\text{Page size} = 4\text{KB} = 2^{12}$$

$$\text{Pages} = 256 = 2^8$$

$$\text{frames} = 64 = 2^6$$

(a) logical address

$$= 2^8 \times 2^{12} = 2^{20} = 20 \text{ bits}$$

(b) Physical address :-

$$= 2^6 \times 2^{12} = 2^{18} = 18 \text{ bits}$$

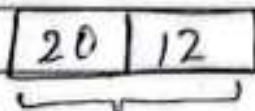
9.10)

$$\text{logical address} = 32 \text{ bit}$$

$$\text{page size} = 4\text{KB} = 2^{12}$$

$$\text{physical memory} = 512 \text{ MB}$$

(a)

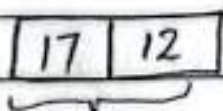


32

so A convential single level page table have 2^{20} entries

(b) in Bytes = $512 \times 1024^2 = 536870912$

$$\log_2(536870912) = 29 \text{ bit}$$



29 bit

so, an inverted table have 2^{17} entries.

Exercises :-

9.13)

200 MB, 15 MB, 185 MB, 75 MB, 175 MB, 80 MB

100 MB	First-fit	Best-fit	Worst-fit
100 MB	15 MB 75 MB	75 MB	
170 MB	80 MB		80 MB
40 MB		15 MB	
205 MB	200 MB	200 MB	15 MB 185 MB
300 MB	185 MB	175 MB 80 MB	200 MB
185 MB	175 MB	185 MB	75 MB

In worst fit 175 MB request cannot be satisfied

- 1) First-fit :- It is fast and practical, good overall usage, but have some fragmentation.
- 2) Best-fit :- Use smallest suitable holes, reduces immediate waste, can create many tiny unusable holes.
- 3) Worst-fit :- leaves medium holes fragmented later often poor long-term strategy here failed

9.21) Page size = 1KB
Page number = ?
offsets = ?

(a) 21205

$$\begin{aligned}\text{Page number} &= \text{logical address} / \text{Page size} \\ &= 21205 / 1024 \\ &= 20\end{aligned}$$

$$\begin{aligned}\text{Offset} &= \text{logical address} \mid \text{Page size} \\ &= 21205 \bmod 1024 \\ &= 725\end{aligned}$$

(b) 164250

$$\begin{aligned}\text{Page No} &= 160 \\ \text{Offset} &= 410\end{aligned}$$

(c) 121357

$$\begin{aligned}\text{Page Number} &= 121357 / 1024 \\ &= 118\end{aligned}$$

$$\begin{aligned}\text{Offsets} &= 121357 \bmod 1024 \\ &= 525\end{aligned}$$

(d) 16479315

$$\begin{aligned}\text{Page Number} &= 16479315 / 1024 \\ &= 16093\end{aligned}$$

$$\begin{aligned}\text{Offset} &= 16479315 \bmod 1024 \\ &= 83\end{aligned}$$

(p) 27253187

$$\begin{aligned}\text{Page Num} &= 27253187 / 1024 \\ &= 26614\end{aligned}$$

$$\begin{aligned}\text{Offset} &= 27253187 \bmod 1024 \\ &= 451\end{aligned}$$

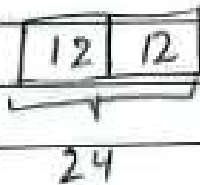
9.22)

virtual address = 24 bit

physical address = 20 bit

Page size = 4KB

(a)



$$\text{offset} = 4KB = 2^{12}$$

a conventional, single level page table have 2^{12} entries $\Rightarrow$ 4KB

(b)



An inverted page table have 2^8 entries

The maximum amount of physical memory in the MPV OS is $2^{20} \Rightarrow 1024KB$

9.23)

logical address space = 2048 pages = 2^{11}
 page size = 4KB = 2^{12}
 frames = 512 = 2^9

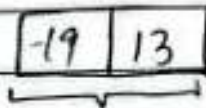
(a) logical address =
 $2^{11} \times 2^{12} = 2^{23} = 23 \text{ bit}$

(b) physical address =
 $2^9 \times 2^{12} = 2^{21} = 21 \text{ bit}$

9.24)

logical address = 32 bit
 physical memory = 1 GB
 page size = 8KB = 2^{13}

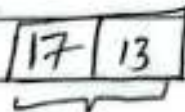
(a)



32 bit

a conventional, single page table have 2^{19} entries $\Rightarrow$ 512 KB

(b)



30 bit

an inverted page table have 2^{17} entries - 128 KB

9.25)

(a) It will take 100 ns for paged memory reference, one for page table (50) & other (50) for memory reference:

$$50 + 50 = 100 \text{ ns}$$

(b) $\alpha = 75\%$ (hit)
0.25 (miss)

TLB = 2 ns

memory access = 50

$$= 0.75 \times 52 + 0.25 \times 102$$

$$= 64.5 \text{ ns}$$

9.28)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main (int argc, char* argv[]) {
```

```
    unsigned long address = atoi(argv[1]);
```

```
    int page-size = 4 * 1024;
```

```
    unsigned long page-number = address / page-size;
```

```
    unsigned long offset = address % page-size;
```

```
    printf("The address %lu contains : \n", address);
```

```
    printf("Page number : %lu \n", page-number);
```

```
    printf("offset : %lu \n", offset);
```

```
    return 0;
```

Chapter 10 :-10.4)CPU cycle time = $1 \mu s$ Extra time for accessing a different page = $1 \mu s$

Page size = 1000 words

Drum speed = 3000 rpm

Transfer rate = 1000000 words/sec

1% instruction access a different page

80% already page in memory

20% cause page faults

50% of replaced pages are modified

(a) 99% instruction does not access another page

normal Time = $1 \mu s$

$$= 0.99 \times 1$$

(b) 80% of 1%.

$$T = 2 \mu s$$

$$0.08 \times 0.01 = 0.008$$

(c) 20% of 1%.

$$0.20 \times 0.01 = 0.002$$

Page fault time $\Rightarrow$

$$(a) 3000 \text{ rpm} = 50 \text{ rotations/sec}$$

$$\text{Time per rotation} = \frac{1}{50} = 0.02 \text{ s} = 20 \text{ ms}$$

$$\text{average delay} = 10 \text{ ms}$$

Date: _____

(b) Transfer time

Page size = 1000 words

Transfer rate = 1,000,000 words/sec

$$\text{Transfer time} = \frac{1000}{1,000,000} = 0.001 \text{ s} = 1 \text{ ms}$$

(c) Page read time

$$T_{\text{read}} = 10 + 1 = 11 \text{ ms}$$

(d) Page write

$$T_{\text{write}} = 0.5 \times 11 = 5.5 \text{ ms}$$

(e) Total page fault time :-

$$T_{\text{total}} \Rightarrow 11 + 5.5 = 16.5 \text{ ms}$$

$$= 16500 \mu\text{s}$$

Probability of page fault time

$$0.002 \times 16500$$

Now, Effective Instruction Time (EIT)

$$\begin{aligned} \text{EIT} &= (0.99)(1) + (0.008)(2) + (0.002)(16500) \\ &= 34.006 \mu\text{s} \end{aligned}$$

10.5)

Page size = 256 byte $\Rightarrow 2^8$ (last two hex digit)
 free frame list $\Rightarrow D \rightarrow E \rightarrow F$

(a) 9EF

virtual address = 12-bit

9	EF
---	----

Page num offset

→ Page table

Page(9) $\Rightarrow$ Page Frame (0)

Physical address

0	EF
---	----

frame offset

Physical address = DEF

(b) 111

1	11
---	----

 virtual address
Page(1) $\rightarrow$ Page frame (2)

211 = physical address

③ 700

7	00
---	----

 virtual address

Page (7) = Page frame (-) not in memory

so free frame list $D \rightarrow E \rightarrow F$

Page frame = D

D00 = physical address

④ 0FF

0	FF
---	----

Page (0) = Page frame (-) not in memory

so free frame list $D \rightarrow E \rightarrow F$

Page frame = D

DFF = physical address

10-8) LRU

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

[illegible]

PF PF PF PF PH PH PF PF PH PH PH PH PF PH PH PH PH PH PH

Page faults = 7

frame = 6

f ₆							6	6	6	6	6	6	6	6	6	6	6	6	6
f ₅							5	5	5	5	5	5	5	5	5	5	5	5	5
f ₄				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f ₃			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f ₁	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	F	F	F	F	H	H	F	F	H	H	H	H	F	H	H	H	H	H	H

Page fault = 7

f_5							5	5	5	5	5	5	7	7	7	7	7	7	7
f_4				4	4	4	4	4	4	4	4	3	3	3	3	3	3	3	3
f_3			3	3	3	3	3	6	6	6	6	6	6	6	6	6	6	6	6
f_2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f_1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	F	F	F	F	H	H	F	F	H	H	H	F	F	H	H	H	H	H	H

Page fault = 8



f_4				4	4	4	4	6	6	6	6	6	7	7	7	7	1	1	1	1
f_3			3	3	3	3	5	5	5	5	5	3	3	3	3	3	3	3	3	3
f_2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f_1	1	1	1	1	1	1	1	1	1	1	1	1	1	6	6	6	6	6	6	6
	F	F	F	F	H	H	F	F	H	H	H	F	F	F	H	H	F	H	H	H

Page fault = 10

f_3			3	3	3	1	1	1	2	2	2	2	2	6	6	6	1	1	1	6
f_2		2	2	2	2	2	2	6	6	6	6	3	3	3	3	3	3	3	3	3
f_1	1	1	1	4	4	4	5	5	5	1	1	1	7	7	7	2	2	2	2	2
	F	F	F	F	H	F	F	F	F	F	H	F	F	F	H	F	F	H	H	F

Ans

Page fault = 15

f_2		2	2	4	4	1	1	6	6	1	1	3	3	6	6	2	2	2	2	6
f_1	1	1	3	3	2	2	5	5	2	2	2	2	7	7	3	3	1	1	3	3
	F	F	F	F	F	F	F	F	F	F	H	F	F	F	F	F	F	H	F	F

Page fault = 18

f_1	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

Page faults = 20

FIFO replacement

f ₇													7	7	7	7	7	7	7
f ₆								6	6	6	6	6	6	6	6	6	6	6	6
f ₅							5	5	5	5	5	5	5	5	5	5	5	5	5
f ₄				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f ₃			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f ₁	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
F F F F H H F F H H H H F H H H H H H																			

Page faults = 7

f ₆								6	6	6	6	6	6	6	6	6	6	6	6
f ₅							5	5	5	5	5	5	5	5	5	5	5	5	5
f ₄				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	3
f ₃			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	2	2
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	1	1	1
f ₁	1	1	1	1	1	1	1	1	1	1	1	1	7	7	7	7	7	7	7
F F F F H H F F H H H H F H H H F F F H																			

Page faults = 10

f ₅							5	5	5	5	5	5	7	7	7	7	7	7	7
f ₄				4	4	4	4	4	4	4	4	3	3	3	3	3	3	3	3
f ₃			3	3	3	3	3	3	3	3	2	2	2	2	2	2	2	2	2
f ₂		2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1
f ₁	1	1	1	1	1	1	1	6	6	6	6	6	6	6	6	6	6	6	6
F F F F H H F F H F F F F H H H H H H H																			

Page faults = 10

f_4				4	4	4	4	4	4	1	1	1	1	1	1	1	1	1	6
f_3			3	3	3	3	3	3	2	2	2	2	2	6	6	2	2	2	2
f_2		2	2	2	2	2	2	6	6	6	6	6	7	7	7	7	7	7	7
f_1	1	1	1	1	1	1	5	5	5	5	5	3	3	3	3	3	3	3	3
F F F F H H F F F F H F F F H F H H H																			

Page fault = 13

f_3		3	3	3	3	3	5	5	5	1	1	1	1	6	6	6	6	6	3	3
f_2		2	2	2	2	1	1	1	2	2	2	7	7	7	7	1	1	1	1	1
f_1	1	1	1	4	4	4	4	6	6	6	6	3	3	3	3	2	2	2	2	6
		F	F	F	F	H	F	F	F	F	F	H	F	F	F	H	F	F	H	F

Page fault = 16

f_2		2	2	4	4	1	1	6	6	1	1	3	3	6	6	2	2	2	3	3
f_1	1	1	3	3	2	2	5	5	2	2	2	2	7	7	3	3	1	1	1	6
F F F F F F F F F F H F F F F F H H F																				

10

Page fault = 18

f_1	1	2	3	4	2	1	5	6	2	1	2	3	7	6	3	2	1	2	3	6
-------	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Page fault = 20

OPTIMAL Replacement :-

f ₇													7	7	7	7	7	7	7
f ₆								6	6	6	6	6	6	6	6	6	6	6	6
f ₅							5	5	5	5	5	5	5	5	5	5	5	5	5
f ₄				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f ₃			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f ₁	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
	F	F	F	F	H	H	F	F	H	H	H	H	F	H	H	H	H	H	H

Page fault = 7

f ₆								6	6	6	6	6	6	6	6	6	6	6	6
f ₅								5	5	5	5	5	5	5	5	5	5	5	5
f ₄				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f ₃			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f ₁	1	1	1	1	1	1	1	1	1	1	1	1	1	7	7	7	7	7	7
	F	F	F	F	H	H	F	F	H	H	H	H	F	H	H	H	F	H	F

Page fault = 9

f ₅							5	5	5	5	5	5	5	5	5	5	5	5	5
f ₄				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f ₃			3	3	3	3	3	6	6	6	6	6	6	6	6	6	6	6	6
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	7	7	7
f ₁	1	1	1	1	1	1	1	1	1	1	1	1	1	3	3	3	3	3	3
	F	F	F	F	H	H	F	F	H	H	H	F	F	H	H	F	F	H	F

Page fault :- 10

Date:

f_4				7	7	7	4	4	4	4	4	4	4	4	4	4		
f_3			3	3	3	3	6	6	6	6	6	6	6	6	2	2	2	
f_2		2	2	2	2	2	2	2	2	2	2	7	7	7	7	7	7	
f_1	1	1	1	1	1	1	1	1	1	1	3	3	3	3	3	1	6	
	F	F	F	F	H	H	F	H	H	H	F	F	H	H	F	F	H	F

Page fault = 11

f_3	3	3	3	3	3	5	5	5	5	5	5	5	6	6	6	1	1	1	6
f_2	2	2	2	2	2	2	2	2	2	2	2	7	7	7	2	2	2	2	2
f_1	1	1	1	4	4	1	1	6	6	1	1	3	3	3	3	3	3	3	3
F	F	F	F	H	F	F	F	H	F	H	F	F	F	H	F	F	H	H	F

Page fault = 14

f_2		2	2	2	2	2	2	2	2	2	2	2	7	6	6	2	2	2	3	3	
f_1	1	1	3	4	4	1	5	6	6	1	1	3	3	3	3	3	1	1	1	6	
		F	F	F	F	H	F	F	F	H	F	H	F	F	F	H	F	F	H	F	F

Page fault = 15

4.1

Page fault :- 20

10.9)

7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 7, 1, 0, 5, 4, 6, 2, 3, 0, 1

f ₃	LRU																				
f ₆	f ₃			3	3	3	5	5	5	6	6	6	6	0	0	0	6	6	6	0	0
f ₅	f ₂		2	2	2	2	2	2	4	4	4	4	1	1	1	4	4	4	3	3	3
f ₁	f ₁	7	7	7	1	1	1	3	3	3	7	1	7	7	5	5	5	2	2	2	1
f ₃		F	F	F	F	H	F	F	F	F	H	F	F	F	F	F	F	F	F	F	

Page fault = 18

FIFO Replacement =

f_3			3	3	3	3	3	4	4	4	4	1	1	1	4	4	4	3	3	3
f_2		2	2	2	2	5	5	5	5	7	7	7	7	5	5	5	2	2	2	1
f_1	7	7	7	1	1	1	1	1	6	6	6	6	0	0	0	6	6	6	0	0
	F	F	F	F	H	F	H	F	F	F	H	F	F	F	F	F	F	F	F	F

Page fault = 17

Optimal Replacement :-

f ₃	f ₂	f ₁			3	3	3	3	3	4	6	7	7	7	0	0	0	0	0	0	0	0
f ₂			2	2	2	2	5	5	5	5	5	5	5	5	5	5	4	6	2	3	3	3
f ₁			7	7	7	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
			F	F	F	F	H	F	H	F	F	F	H	H	F	H	F	F	F	F	H	H

Page fault = 12

OP < LRU < FI

10.18.)Page size = 256 bytes = 2^8

Free-frame list = 9 → F → D

(a) 0x2A1

virtual address = 2A1

physical address = AA1

(b) 0x4E6

virtual = 4E6

physical = 9E6

(c) 0x94A

virtual address = 94A

physical address = 14A

(d) 0x316

virtual address = 316

physical address = F16

8

10-20)

11123456

virtual address space = 2^{32} bytes $\rightarrow$ 32 bit virtual
 physical address space = 2^{22} bytes $\rightarrow$ 22 bit physical
 page size = 4096 bytes = 2^{12} bytes $\rightarrow$ 12 bit offset

virtual page number = 20 bits

frame number = 10 bits

virtual address $\Rightarrow$

virtual page number = 11123

offset = 456

Then the MMU takes VPN and search it in page table (or TLB) if PFN^{not} exists in memory then page fault will be generated. Otherwise it combine
 PFN + offset = physical address.

Hardware role $\rightarrow$ splits virtual address

$\rightarrow$ look TLB

$\rightarrow$ performs page table lookup (if TLB miss)

$\rightarrow$ forms physical address

$\rightarrow$ Triggers page fault if page missing

Software role $\rightarrow$ Only runs on page fault.

- find page table on disk

- Allocate free frame

- Update page table

- Bring page in memory

- restarts instructions.

Date: _____

10.21)

Memory access time = 100 ns

page fault for empty frame = 8 ms

page fault dirty page = 20 ms

page replaced is modified = 70%.

page replaced not modified = 30%.

EAT $\leq$ 200

Sol:

page fault service Time :-

$$= 0.3(8) + 0.7(20)$$

$$= 0.3(8000000) + 0.7(20000000)$$

$$= 16400000 \text{ ns}$$

$$\text{EAT} = (1-p) \times 100 + p \times 16400000$$

$$(1-p) \times 100 + p \times 16400000 \leq 200$$

$$100 - 100p + 16400000p \leq 200$$

$$100 + 16399900p \leq 200$$

$$16399900p \leq 100$$

$$p \leq \frac{100}{16399900}$$

$$16399900$$

$$p \leq 6.1 \times 10^{-6} \%$$

10-22)

Page size = 4096 bytes = 2^{12} bytes $\Rightarrow$ 12-bit
virtual, physical address = 16-bit

offsets = 12 bit
virtual page number = 4 bit
page frame number = 4 bit

(a)

i) VPN = 6
offset = 21C

Page (6) $\Rightarrow$ Page Frame (8)

Physical address = 821C
Page (6) referenced bit = 1

ii) VPN = 0F
offset = 0A3

Page (15) $\Rightarrow$ Page frame (2)

Physical address = 20A3
Page (15) referenced bit = 1

iii) VPN = B
offset = C1A
PFN = 4

physical address = 4C1A
page (11) referenced bit = 1

Date: _____

iv) ~~0x5B~~
0x5BAA

VPN = 5

offset = BAA

PFN = D (13)

physical address = DBAA

page (5) referenced bit = 1

v) 0x0BA1

VPN = 0

offset = BA1

PFN = 9

physical address = 9BA1

page (0) referenced bit = 1

(b) logical address = 0xEA20

(c) The localized LRU page-replacement selects a victim frame from the set of page frames currently allocated to this specific. It will look at the frames active in the page table 9, 10, 15, 6, 13, 8, 12, 7, 5, 4, 1, 0, 2 and replace the one that has not been used for the longest period of time, rather than pulling from the system-wide global pool of frames.

10.24)

i) 2, 6, 9, 2, 4, 2, 1, 7, 3, 0, 5, 2, 1, 2, 9, 5, 7, 3, 8, 5

FIFO

f_3			9	9	9	9	1	1	1	0	0	0	1	1	9	9	1	7	8	8
f_2		6	6	6	6	2	2	2	3	3	3	2	2	2	2	2	2	3	3	3
f_1	2	2	2	2	4	4	4	7	7	7	5	5	5	5	5	5	7	7	7	5
	F	F	F	H	F	F	F	F	F	F	F	F	F	H	F	H	F	F	F	F

Page fault = 18

LRU

f_3			9	9	9	9	1	1	1	0	0	0	1	1	1	5	5	5	8	8
f_2		6	6	6	4	4	4	7	7	7	5	5	5	5	9	9	9	3	3	3
f_1	2	2	2	2	2	2	2	2	3	3	3	2	2	2	2	2	7	7	7	5
	F	F	F	H	F	H	F	F	F	F	F	F	F	H	F	F	F	F	F	F

Page fault = 17

Optimal

f_3			9	9	9	9	9	7	3	0	5	5	5	5	5	5	5	5	5	5
f_2		6	6	6	4	4	1	1	1	1	1	1	1	1	9	9	9	3	3	3
f_1	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	8	8	8
	F	F	F	H	F	H	F	F	F	F	H	H	H	F	H	F	F	F	F	H

Page fault = 9

Date: _____

ii) 0, 6, 3, 0, 2, 6, 3, 5, 2, 4, 1, 3, 0, 6, 1, 4, 2, 3, 5, 7

FIFO

f ₃			3	3	3	3	3	3	3	4	4	4	0	0	0	4	4	4	5	5
f ₂		6	6	6	6	6	6	5	5	5	5	3	3	3	1	1	1	3	3	3
f ₁	0	0	0	0	2	2	2	2	2	2	1	1	1	6	6	6	2	2	2	7
	F	F	F	H	F	H	H	F	H	F	F	F	F	F	F	F	F	F	F	F

Page fault = 16

LRU

f ₃			3	3	3	6	6	6	2	2	2	3	3	3	1	1	1	3	3	3
f ₂		6	6	6	2	2	2	5	5	5	1	1	1	6	6	6	2	2	2	7
f ₁	0	0	0	0	0	0	3	3	3	4	4	4	0	0	0	4	4	4	5	5
	F	F	F	H	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

Page fault = 19

Optimal

f ₃			3	3	3	3	3	3	3	3	3	3	0	6	6	6	2	2	2	7
f ₂		6	6	6	6	6	6	5	5	4	4	4	4	4	4	4	4	4	3	3
f ₁	0	0	0	0	2	2	2	2	2	2	1	1	1	1	1	1	1	1	5	5
	F	F	F	H	F	H	H	F	H	F	F	H	F	F	H	H	F	F	F	F

Page fault = 13

Date: _____

iii) 3, 1, 4, 2, 5, 4, 1, 3, 5, 2, 0, 1, 1, 0, 2, 3, 4, 5, 0, 1

FIFO

f_3			4	4	4	4	1	1	1	1	0	0	0	0	0	0	4	4	4	1
f_2		1	1	1	5	5	5	5	5	2	2	2	2	2	2	3	3	3	0	0
f_1	3	3	3	2	2	2	2	3	3	3	3	1	1	1	1	1	1	5	5	5
	F	F	F	F	F	H	F	F	H	F	F	F	H	H	H	F	F	F	F	F

Page fault = 15

LRU

IRU																				
f_3			4	4	4	4	4	4	5	5	5	1	1	1	1	3	3	3	0	0
f_2		1	1	1	5	5	5	3	3	3	0	0	0	0	0	0	4	4	4	1
f_1	3	3	3	2	2	2	1	1	1	2	2	2	2	2	2	2	2	5	5	5
	F	F	F	F	F	H	F	F	F	F	F	H	H	H	F	F	F	F	F	F

Page fault = 16

Optimal

f ₃			4	4	4	4	4	3	3	3	0	0	0	0	0	0	0	0	0
f ₂		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
f ₁	3	3	3	2	5	5	5	5	5	2	2	2	2	2	2	3	4	5	5
	F	F	F	F	F	H	H	F	H	F	F	H	H	H	H	F	F	F	H

Page fault = 11

Date: _____

iv) 4, 2, 1, 7, 9, 8, 3, 5, 2, 6, 8, 1, 0, 7, 2, 4, 1, 3, 5, 8

FIFO :-

f ₃			1	1	1	8	8	8	2	2	2	1	1	1	2	2	2	3	3	3
f ₂		2	2	2	9	9	9	5	5	5	8	8	8	7	7	7	1	1	1	8
f ₁	4	4	4	7	7	7	3	3	3	6	6	6	0	0	0	4	4	4	5	5
	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

Page fault = 20

LRU :

f ₃			1	1	1	8	8	8	2	2	2	1	1	1	2	2	2	3	3	3
f ₂		2	2	2	9	9	9	5	5	5	8	8	8	7	7	7	1	1	1	8
f ₁	4	4	4	7	7	7	3	3	3	6	6	6	0	0	0	4	4	4	5	5
	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

Page fault = 20

Optimal

f ₃			1	1	1	1	8	5	5	6	6	1	1	1	1	1	1	3	3	3
f ₂		2	2	2	2	2	2	2	2	2	2	2	2	2	2	4	4	4	5	5
f ₁	4	4	4	7	9	8	8	8	8	8	8	0	7	7	7	7	7	1	8	
	F	F	F	F	F	F	F	H	F	H	F	F	F	H	F	H	F	F	F	

Page fault = 16

Date: _____

iv) 4, 2, 1, 7, 9, 8, 3, 5, 2, 6, 8, 1, 0, 7, 2, 4, 1, 3, 5, 8

FIFO :-

f_3			1	1	1	8	8	8	2	2	2	1	1	1	2	2	2	3	3	3
f_2		2	2	2	9	9	9	5	5	5	8	8	8	7	7	7	1	1	1	8
f_1	4	4	4	7	7	7	3	3	3	6	6	6	0	0	0	4	4	4	5	5
	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

Page fault = 20

LRU :

f_3			1	1	1	8	8	8	2	2	2	1	1	1	2	2	2	3	3	3
f_2		2	2	2	9	9	9	5	5	5	8	8	8	7	7	7	1	1	1	8
f_1	4	4	4	7	7	7	3	3	3	6	6	6	0	0	0	4	4	4	5	5
	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F	F

Page fault = 20

Optimal

f_3			1	1	1	1	3	5	5	6	6	1	1	1	1	1	3	3	3	
f_2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	4	4	4	5	5
f_1	4	4	4	7	9	8	8	8	8	8	8	8	0	7	7	7	7	7	1	8
	F	F	F	F	F	F	F	H	F	H	F	F	F	H	F	H	F	F	F	

Page fault = 16

Date: _____

iv) 0, 1, 2, 3, 4, 4, 3, 2, 1, 0, 0, 1, 2, 3, 4, 4, 3, 2, 1, 0

FIFO

f ₃			2	2	2	2	2	2	1	1	1	1	1	3	3	3	3	3	0
f ₂		1	1	1	4	4	4	4	4	4	4	4	2	2	2	2	2	1	1
f ₁	0	0	0	3	3	3	3	3	3	0	0	0	0	0	4	4	4	4	4
	F	F	F	F	F	H	H	H	F	F	H	H	F	F	F	H	H	F	F

Page fault = 12

LRU

f ₃			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f ₂		1	1	1	4	4	4	4	1	1	1	1	1	4	4	4	4	1	1
f ₁	0	0	0	3	3	3	3	3	3	0	0	0	0	3	3	3	3	3	0
	F	F	F	F	F	H	H	H	F	F	H	H	H	F	F	H	H	H	F

Page fault = 11

Optimal

f ₃			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	1	1
f ₂		1	1	1	4	4	4	4	1	1	1	1	1	1	4	4	4	4	0
f ₁	0	0	0	3	3	3	3	3	3	0	0	0	0	3	3	3	3	3	3
	F	F	F	F	F	H	H	H	F	F	H	H	H	F	F	H	H	H	F

Page fault = 11

Date: _____

10.35)

(a) A page replacement algorithm can be designed by maintaining a counter for each page frame to track how heavily that frame is being used. The algorithm attempts to distribute heavily used pages evenly across memory to reduce page faults.

- 1) Initially all the counters are initialized to 0 because no page is loaded.
- 2) When a page is loaded into a page frame or whenever a page is referenced the counter increases.
- 3) When ever a page is replaced from a page frame it's counter decreases.
- 4) To replace a page, we can search for the page frame with the smallest counter.

(b) 1, 2, 3, 4, 5, 3, 4, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4, 5, 4, 2

f_4				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f_3			3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
f_2		2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
f_1	1	1	1	1	5	5	5	1	6	7	8	7	8	9	7	8	9	5	5	5	5
	F	F	F	F	F	H	H	F	F	F	F	F	F	F	F	F	F	F	H	H	H

$$f_1 \text{ count} = \cancel{X} 2$$

$$f_2 \text{ count} = \cancel{X} 2$$

$$f_3 \text{ count} = \cancel{X} 2$$

$$f_4 \text{ count} = \cancel{X} \cancel{X} 34$$

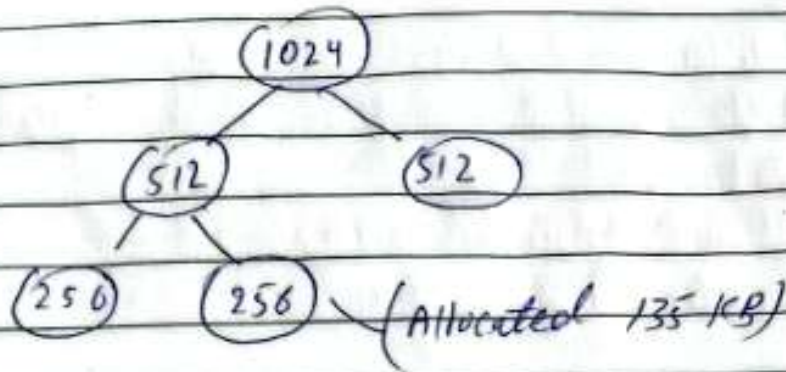
$$\text{Page fault} = 16$$

(c) Optimal

f_4				4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4
f_3			3	3	3	3	3	3	3	7	7	7	7	7	7	7	5	5	5	5	5
f_2		2	2	2	5	5	5	5	5	3	3	3	3	9	9	9	9	7	7	7	2
f_1	1	1	1	1	5	5	5	5	6	6	8	8	8	8	8	8	8	8	8	8	8
	F	F	F	F	F	H	H	H	F	F	F	H	H	F	H	H	H	F	H	H	F

$$\text{Page fault} = 11$$

Modified:

10.1Array size = $100 \times 100 = 10,000$ integersData layout: $A[0][0], A[0][1], \dots$ Element size = 1 integer = 4 bytes = $4 \times 100 = 400$ bytesPage size = 200 words/elements $\Rightarrow 2 \times 100 = 200$ elements

Memory allocation:-

Page 0: Process code

Page 1: array elements $A[0][0]$ to $A[1][99]$ Page 2: Array elements $A[2][0]$ to $A[3][99]$

frames = 3

(a) array down columns

LCU * With only 2 frames available for data, the system constantly evicts the oldest page before it can ever be reused for the next column index. No data pages are preserved for the next outer loop iteration.

Page fault = occur on every single element access.

Total page fault = $100 \times 100 = 10,000$

Date: _____

(b) across row ($A[0][0]$).

Page fault occur only when the loop crosses a page boundary (every 200 elements / every 2 rows). The page stays in memory while its 200 elements are processed sequentially.

Page faults = $\frac{10,000 \text{ total elements}}{200 \text{ elements per page}}$

$$\text{Total page faults} = \frac{10,000}{200} = 50$$